

Makron

Man kan även använda preprocessorn för att skapa *makron* av olika slag. Även denna funktionalitet brukar numera ersättas av en nyare konstruktion i C++. För att skapa makron använder man sig av `#define`, precis som då man skapar konstanter. En makro är egentligen en liten funktion som man kan ge argument åt. Skillnaden mellan ett makro och en vanlig funktion är att koden för ett makro placeras direkt på platsen för anropet, d.v.s. det blir inget funktionsanrop överhuvudtaget. Detta är en aning snabbare än att ha en separat funktion, men i längden även mera utrymmeskrävande, eftersom samma makro kan vara utspritt på många ställen i ett program. Ett makro som beräknar kvadraten på det givna argumentet kan se ut så här:

```
#define KVADRAT(X) X * X
```

Ett program kan sedan använda denna definition på följande sätt:

```
#include <iostream>

#define KVADRAT(X) X * X

int main () {
    cout << "Kvadraten på 5 = " << KVADRAT(5) << endl;
}
```

Vi har nu definierat en makro `KVADRAT` som tar ett argument. Texten `X * X` är den text som `KVADRAT` ersätts med, förstås med `X` ersatt med det värde som man skrivit in. Kompilatorn ser egentligen i vårt fall följande rad:

```
cout << "Kvadraten på 5 = " << 5 * 5 << endl;
```

Man bör vara försiktig då man definierar makron. T.ex. följande skulle inte fungera med vår makrodefinition:

```
cout << "Kvadraten på 2 + 3 = " << KVADRAT(2 + 3) << endl;
```

Det skulle bli följande text:

```
cout << "Kvadraten på 2 + 3 = " << 2 + 3 * 2 + 3 << endl;
```

vilket inte är vad vi avsåg. Vi kan förbättra vår makrodefinition med hjälp av parenteser för att tvinga fram den önskade evalueringsordningen:

```
#define KVADRAT2(X) ((X) * (X))
```

Denna definition fungerar för diverse aritmetiska uttryck. Man kan även skapa makron som tar flera än ett argument. Ett klassiskt exempel är ett makro som ger ut det större av två värden:

```
#define MAX(X,Y) (X > Y ? X : Y)
```

Även här använder vi parenteser för att bättre gruppera evalueringen. Vi använder oss av ?-operatoren för att utföra jämförelsen. Med C++ kan man skriva betydligt mera flexibla makro-liknande funktioner med s.k. *inline*-kod. Dylig kod placeras även av kompilatorn direkt på anropets plats, så inget funktionsanrop görs.

Nackdelar med makron

Det finns alltså några ganska allvarliga begränsingar och nackdelar med att använda makron. Man bör noga överväga ifall fördelarna överväger nackdelarna. I de flesta fall är det enklare att definiera en likadan funktion (eller metod) och låta kompilatorn sköta optimeringen av koden.

De olika nackdelarna är:

- *kräver mera minne* eftersom samma kod finns på många ställen i programmet.
- *svåra att konstruera rätt* så att man får parenteser o.dyl. på de ställen där de behövs. Vanligen märker man dock inte dessa ställen förrän man spenderat tid med att söka efter logiska fel i programmet.
- *ingen typchecking* vilket betyder att man kan t.ex. skicka en sträng till ett makro som utför en aritmetisk operation. Detta godkänner förstås inte kompilatorn, men det kan vara svårt att veta exakt vad som gått fel.

[Förutgående](#)
Konstanter

[Hem](#)
[Upp](#)

[Nästa](#)
Multipel inkludering